

ETHICAL HACKING LAB REPORT

Fernando Gesa

May 2026

-

HTTP Proxy Interception & Traffic Analysis Using Burp Suite Community Edition v2025.10.6

DISCLAIMER: This report documents a lab exercise performed in a fully isolated virtual machine environment. All activities were conducted for educational purposes only. Performing these techniques against real systems without explicit written authorisation is illegal and unethical

1. Objective

The objective of this lab exercise was to configure Burp Suite Community Edition as an intercepting HTTP proxy, route browser traffic through it, and observe how HTTP requests — including form POST submissions containing credentials — can be captured and inspected in real time.

This builds directly on the previous SET Credential Harvester lab by demonstrating that even traffic directed at a cloned page can be observed and manipulated at the proxy layer, which is a core skill for web application penetration testing.

2. Lab Environment

The following environment was used for this exercise:

Component	Details
Attacker Machine	Kali Linux (VM) — giuann66@Giuann
Browser	Firefox 140.0 (configured as proxy client)
Proxy Tool	Burp Suite Community Edition v2025.10.6
Proxy Listener	127.0.0.1:8080 (default Burp listener)
Target Server	http://10.0.2.15/ — SET Credential Harvester (local VM)
Network	Host-only / isolated — no external internet access
CA Certificate	PortSwigger CA (cacert.der) imported into Firefox

3. Methodology

The lab was completed in five phases: launching Burp Suite, configuring Firefox to route traffic through the Burp proxy, installing the Burp CA certificate to enable HTTPS interception, enabling the intercept, and finally capturing a credential POST submission.

Phase 1 — Launch Burp Suite

Burp Suite Community Edition was launched from the Kali terminal using 'sudo burpsuite'. A temporary in-memory project was created with default Burp settings, and the Dashboard confirmed the live passive crawl task was active.

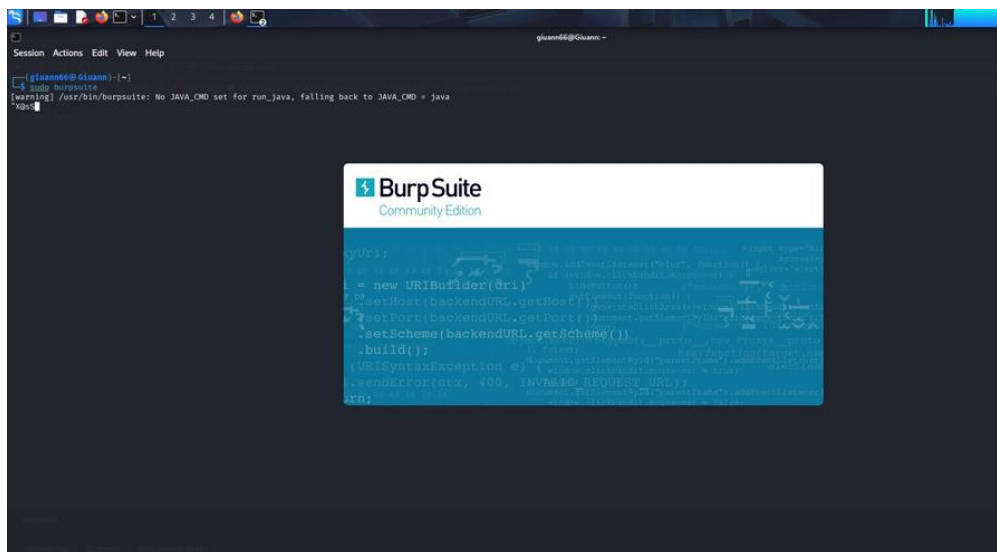


Figure 1 — Launching Burp Suite from the Kali terminal

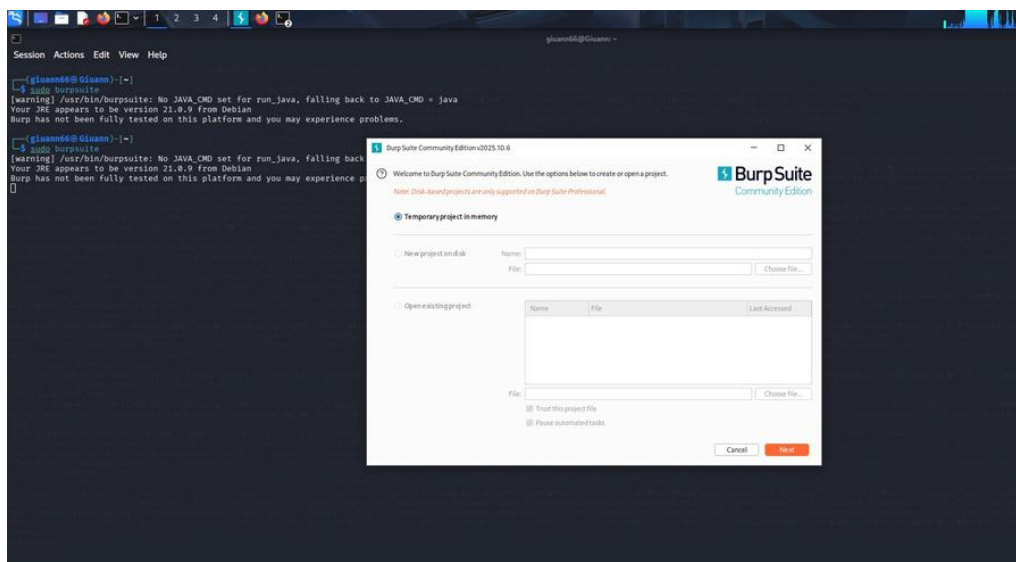


Figure 2 — Creating a temporary in-memory project

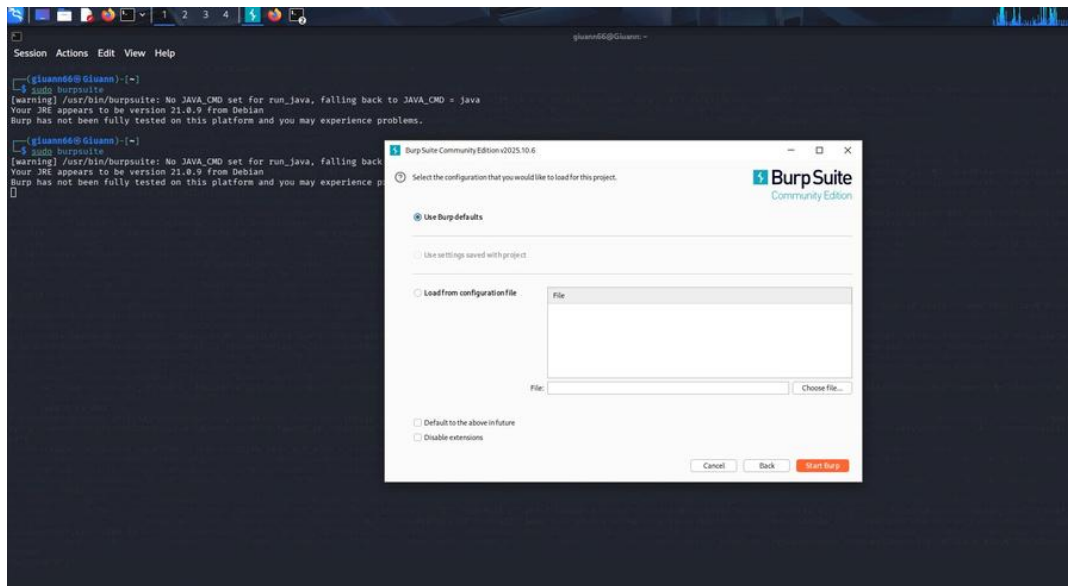


Figure 3 — Selecting Burp default configuration

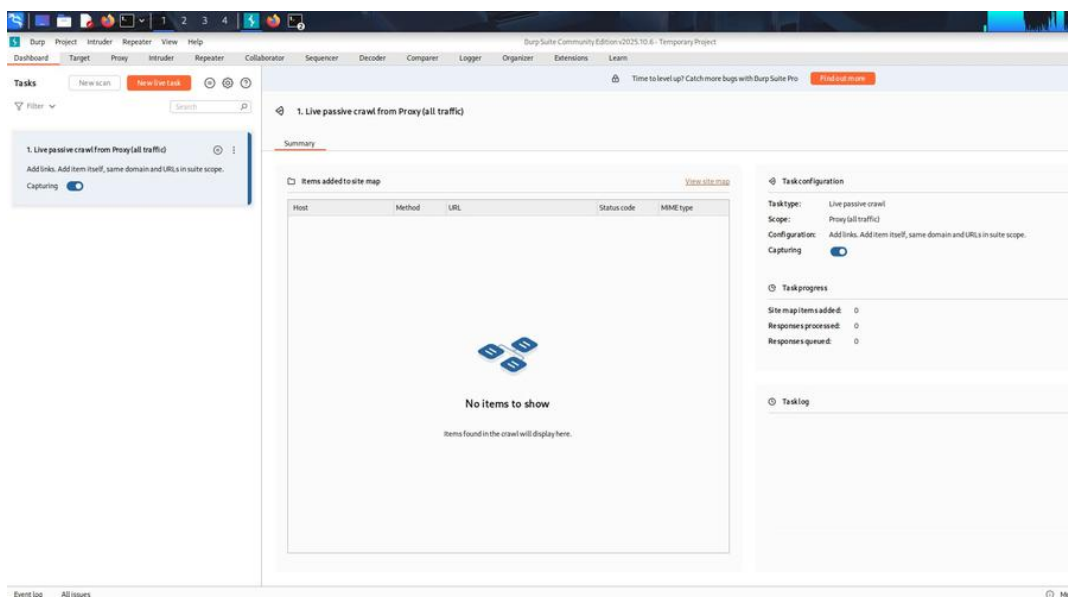


Figure 4 — Burp Suite Dashboard showing live passive crawl task

Phase 2 — Configure Firefox Proxy

Firefox was configured to route all HTTP/HTTPS traffic through Burp's listener at 127.0.0.1:8080. This was done via Firefox Settings > Network Settings > Manual proxy configuration. Without this step, Burp cannot intercept browser traffic.

Initially browsing to google.com produced a 'proxy server is refusing connections' error, confirming Firefox was correctly pointed at Burp but Burp's intercept was still off — the expected behaviour at this stage.

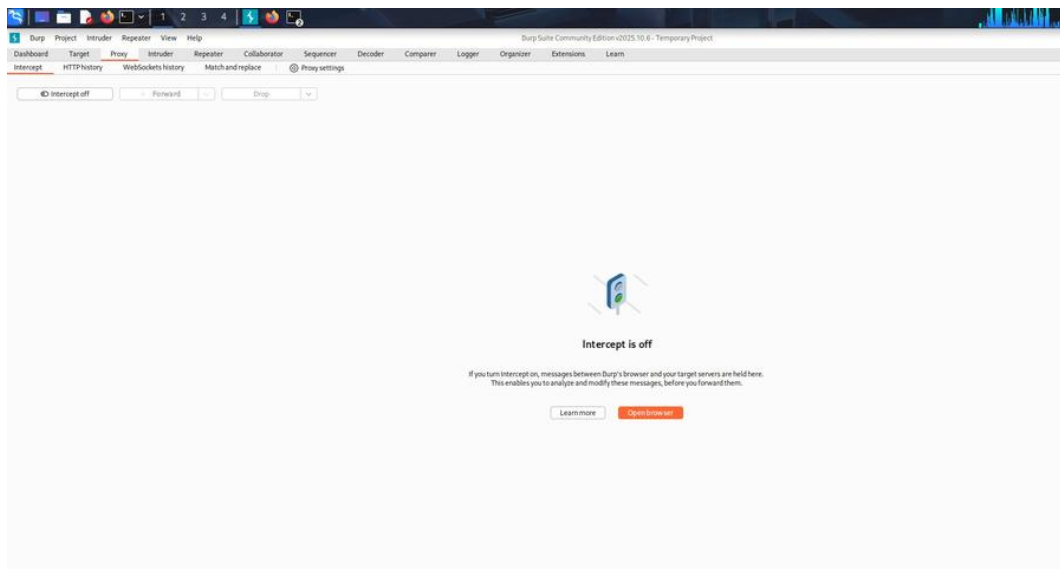


Figure 5 — Burp Proxy tab showing Intercept is off

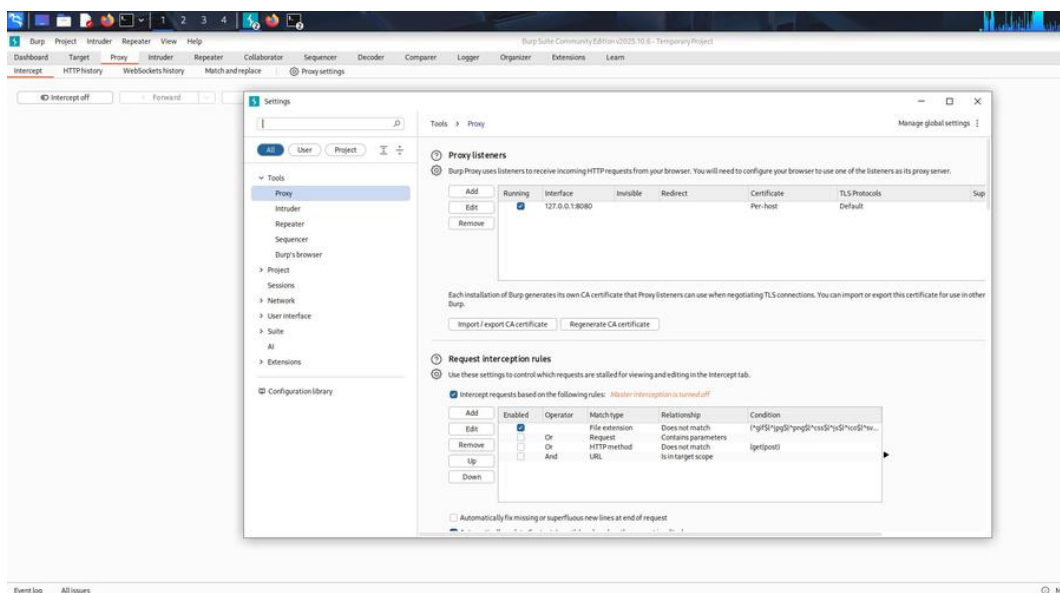


Figure 6 — Burp Proxy listener running on 127.0.0.1:8080

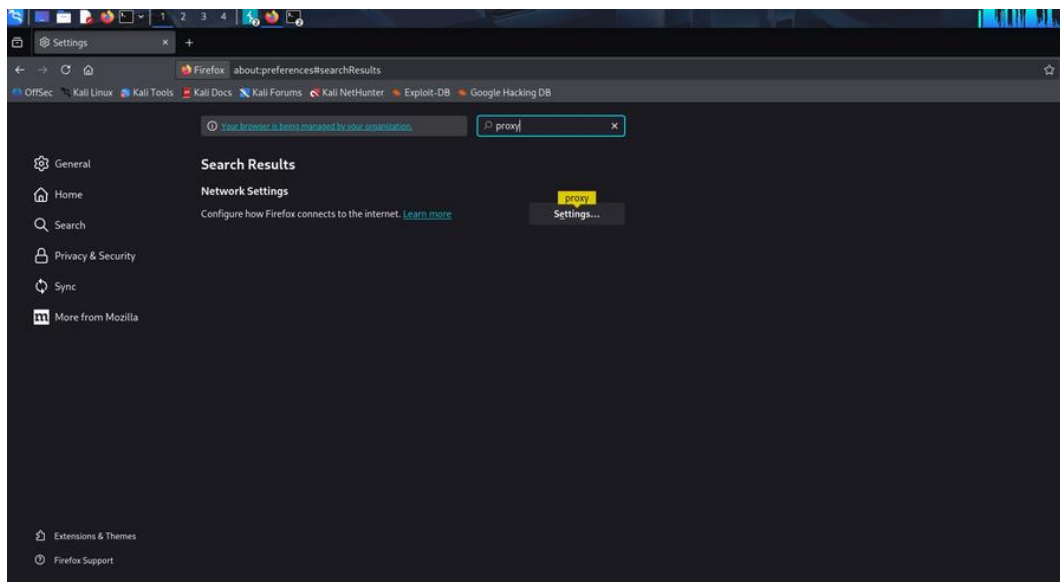


Figure 7 — Firefox network proxy settings pointed at 127.0.0.1:8080

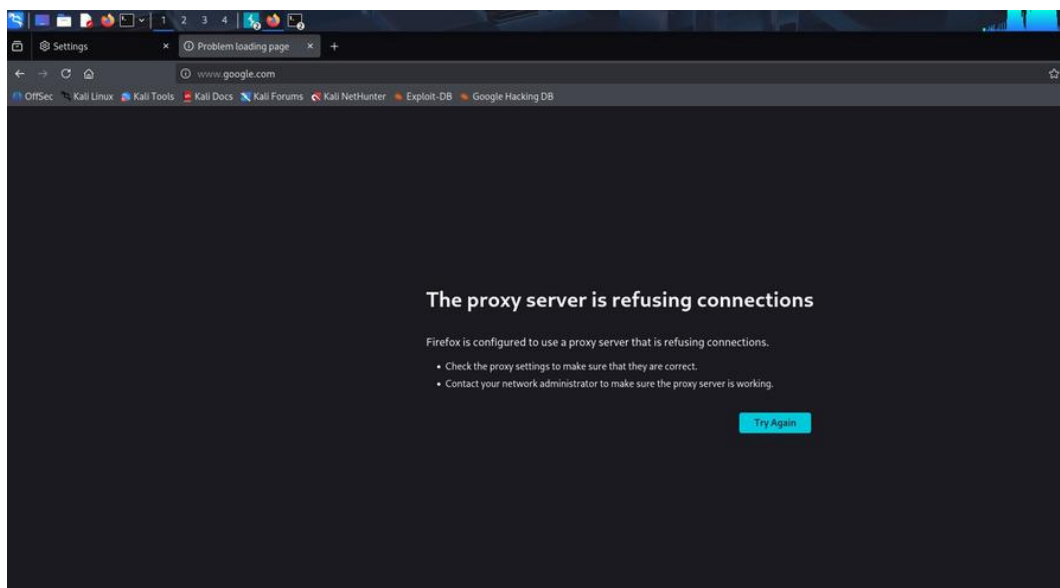


Figure 8 — Expected proxy error confirms Firefox is routing through Burp

Phase 3 — Install Burp CA Certificate

To intercept HTTPS traffic without triggering browser security warnings, Burp's CA certificate must be trusted by the browser. The certificate was downloaded by browsing to <http://burp> (Burp's local portal) and saving `cacert.der`. It was then imported into Firefox via Settings > Certificates > Authorities > Import, trusting PortSwigger CA for both website and email identity.

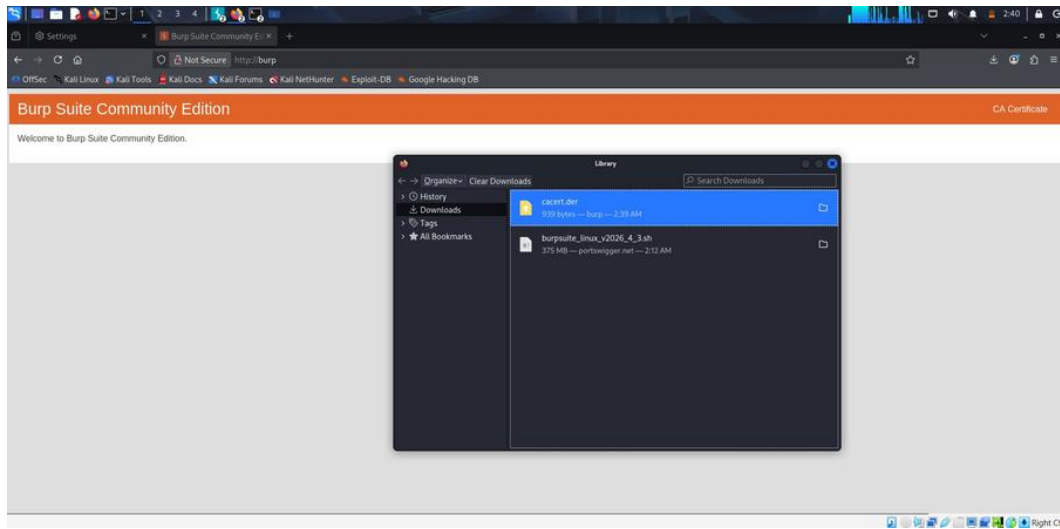


Figure 9 — Downloading `cacert.der` from Burp's local portal

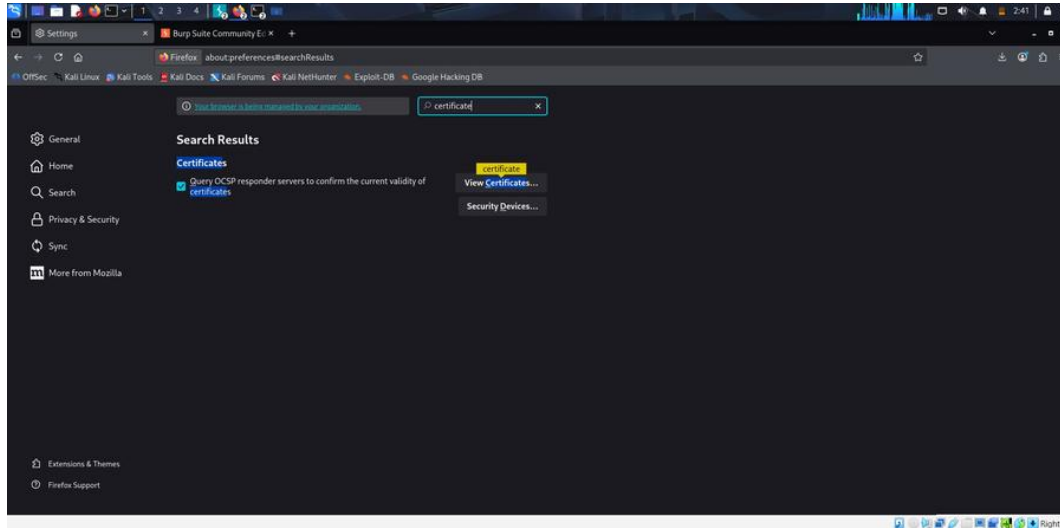


Figure 10 — Navigating to Firefox certificate management

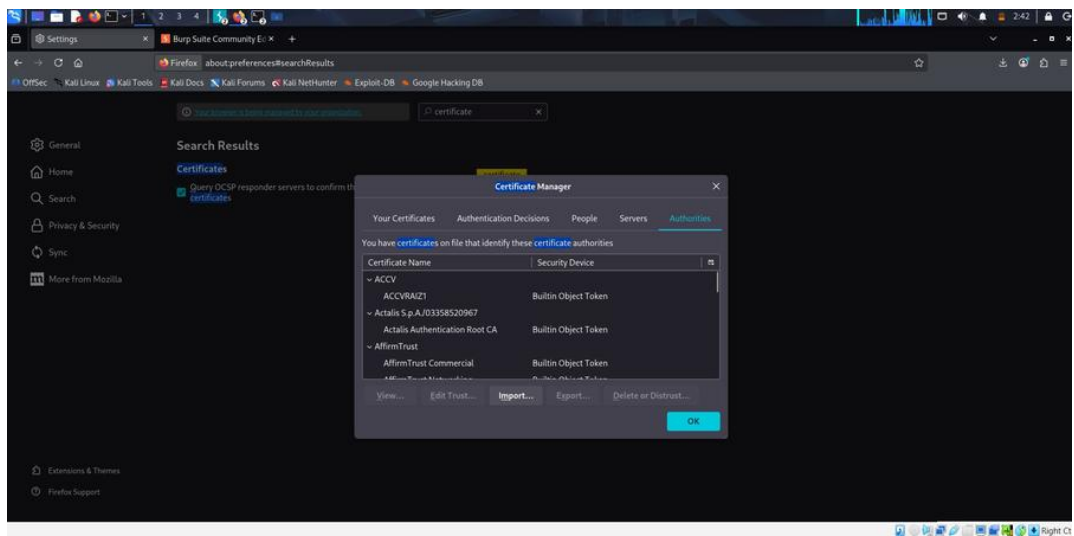


Figure 11 — Firefox Certificate Manager, Authorities tab

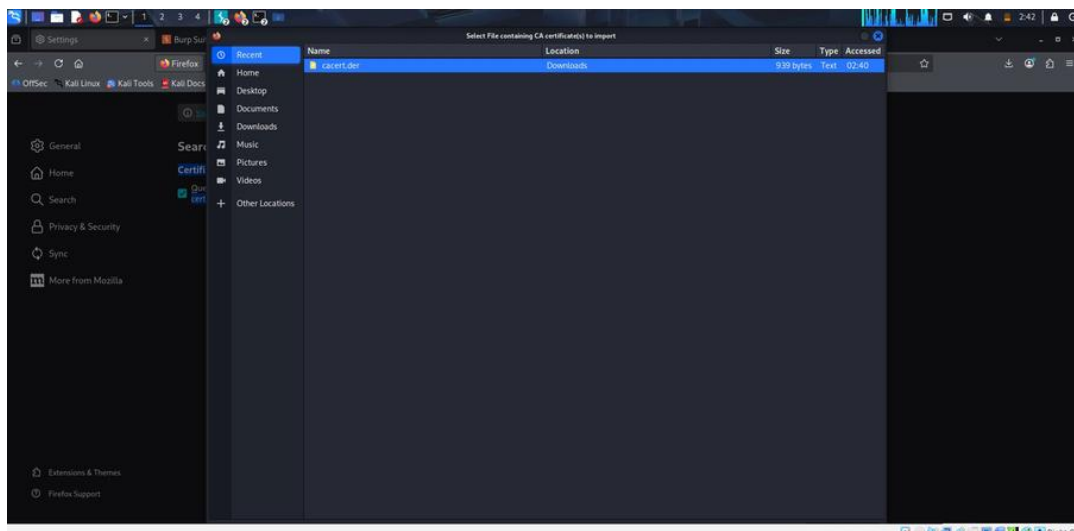


Figure 12 — Selecting cacert.der for import

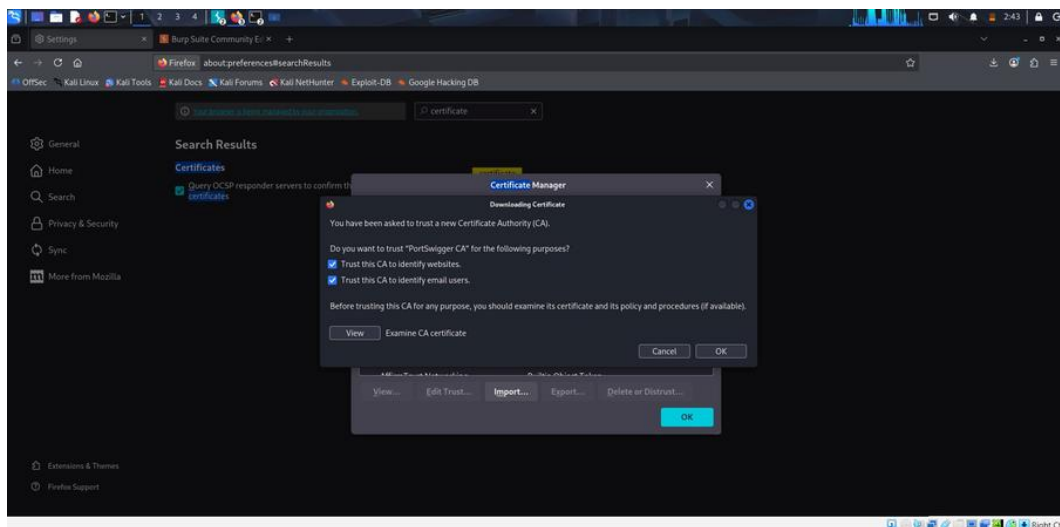


Figure 13 — Trusting PortSwigger CA for website identification

Phase 4 — Enable Intercept & Browse Target

Intercept was turned on in Burp's Proxy tab. Browsing to <http://10.0.2.15/> (the SET Credential Harvester cloned login page running locally) caused Burp to capture the initial GET request, which was visible in the Intercept tab along with the full HTTP headers. The request was forwarded to load the page in the browser.

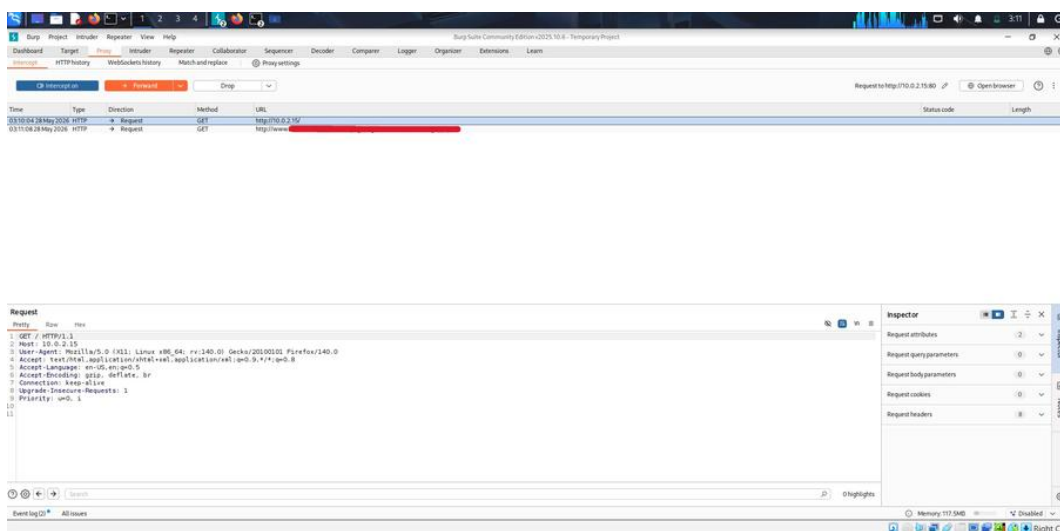


Figure 14 — Burp intercept on, capturing GET request to 10.0.2.15

Phase 5 — Capture POST Credentials

Test credentials were entered into the cloned login form (Cognome: Smith, Nome: John, Password: mypassword22) and the Accedi (Login) button was clicked. Burp intercepted the resulting HTTP POST request before it reached the server.

The full POST body was visible in the Request pane, highlighted in yellow, showing all form parameters in plaintext including the password field.

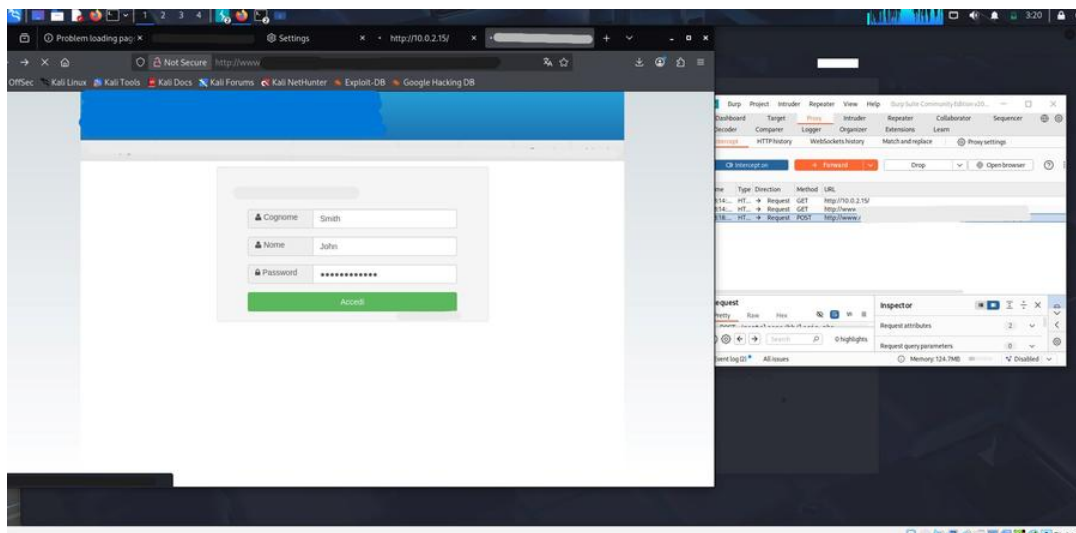


Figure 15 — Credentials entered into the cloned login page with Burp intercepting

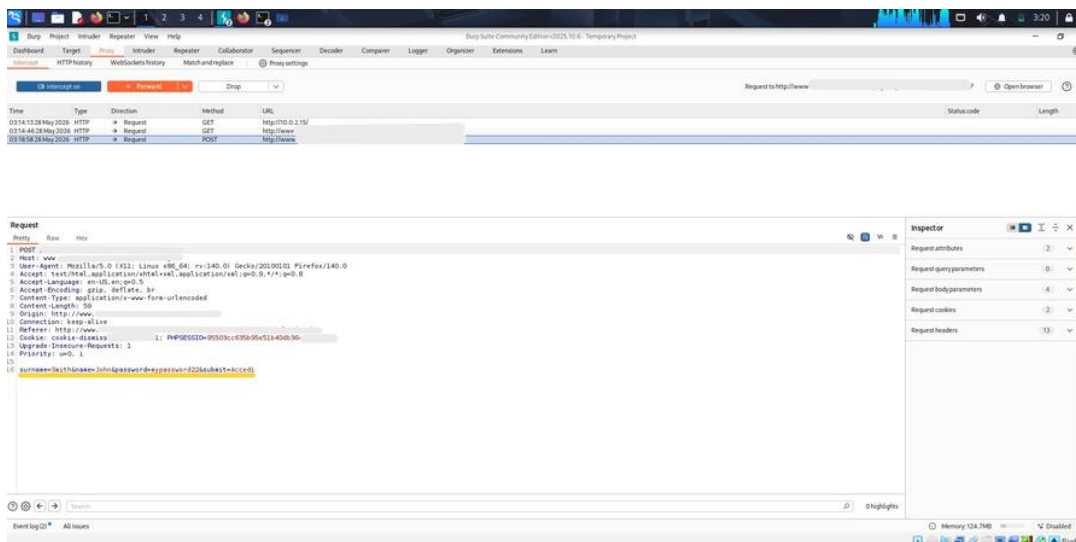


Figure 16 — Burp capturing the POST body including plaintext password

4. Key Commands & Configuration

The following commands and configuration steps were used during the lab:

```
$ sudo burpsuite # Launch Burp Suite
# Firefox: Settings > Network Settings > Manual proxy
# HTTP Proxy: 127.0.0.1 Port: 8080
# Navigate to: http://burp # Download cacert.der
# Firefox: Settings > search 'certificate' > View Certificates
# Authorities > Import > cacert.der > Trust for websites
# Burp: Proxy > Intercept > Intercept on
# Browse: http://10.0.2.15/ # Target cloned page
```

5. Findings

Burp Suite successfully intercepted all HTTP traffic between Firefox and the target server at 10.0.2.15. The key finding was the capture of the credential POST submission, where all form parameters were visible in plaintext in the request body:

```
POST /login/auth.php HTTP/1.1
Host: 10.0.2.15
Content-Type: application/x-www-form-urlencoded
Cookie: cookie-dismiss=1; PHPSESSID=95503cc635b95e51b40db36...

surname=Smith&name=John&password=mypassword22&submit=Accedi
```

Key observation: The HTTP POST body was captured in its entirety before being forwarded to the server. This demonstrates that any unencrypted (HTTP) form submission — including credentials — is fully readable at the proxy layer. Burp's Inspector pane also showed 4 request body parameters and 2 cookies were present in the intercepted request.

6. What This Demonstrates

This lab illustrates several important security concepts relevant to web application penetration testing:

- **HTTP vs HTTPS:** Credentials submitted over plain HTTP are readable by anyone in a position to intercept the traffic — including a proxy, a rogue Wi-Fi access point, or an on-path attacker. HTTPS encrypts the transport layer, but a MITM proxy with a trusted CA certificate (like Burp's) can still decrypt it.
- **MITM via CA Trust:** By installing Burp's CA certificate as a trusted authority in the browser, the proxy performs a man-in-the-middle attack on HTTPS connections — presenting its own certificate to the browser while connecting upstream to the real server. This is standard for authorised penetration testing.
- **Full Request Visibility:** Beyond just credentials, Burp exposed session cookies (PHPSESSID), referrer headers, User-Agent strings, and all other request metadata — information that could be used for session hijacking or fingerprinting in a real engagement.
- **Complementing SET:** Used together with the SET Credential Harvester from the previous lab, Burp provides a second layer of credential capture with far more detail — full headers, cookies, and the ability to modify requests before forwarding them.

7. Defensive Recommendations

This exercise highlights several controls organisations should implement to protect against proxy-based interception attacks:

- **Enforce HTTPS Everywhere:** All login forms and sensitive endpoints must use HTTPS with valid certificates. HTTP form submissions should be rejected or redirected.
- **HSTS (HTTP Strict Transport Security):** HSTS prevents browsers from connecting to a site over HTTP after first contact, reducing the window for downgrade attacks.

- **Certificate Transparency & Pinning:** Certificate pinning prevents rogue CA certificates (such as those installed by a corporate proxy or attacker) from being used to intercept traffic to specific domains.
- **User Awareness Training:** Users should be trained to verify HTTPS padlock indicators and be suspicious of any login page served over plain HTTP.
- **Network Monitoring:** Organisations should monitor for unusual proxy configurations on endpoints and for traffic routed through unexpected intermediate hosts.

8. Conclusion

This lab successfully demonstrated the use of Burp Suite Community Edition as an intercepting HTTP proxy in a controlled, isolated virtual environment. The full setup process was completed — from launching Burp, configuring Firefox, installing the CA certificate, enabling intercept, and ultimately capturing a plaintext credential POST submission.

Combined with the SET Credential Harvester lab, this exercise provides a solid foundation in two core social engineering and web proxy techniques used in real-world penetration testing engagements. Both tools complement each other: SET creates the deceptive capture point, while Burp provides deep visibility into the HTTP traffic passing through it.

This report was produced for educational and portfolio purposes. All testing was performed in an isolated virtual lab environment.